

# 概要设计说明书

## 分布式博客系统

作 者： 杨子贤、舒俊杰、张宇、赵紫东

完成日期：2026 年 5 月 8 日

签 收 人： 杨子贤

签收日期： 2026 年 5 月 9 日

# 目录

## 1 引言

### 1.1 编写目的

### 1.2 范围

### 1.3 定义

### 1.4 参考资料

## 2 总体设计

### 2.1 需求规定

### 2.2 运行环境

### 2.3 基本设计概念和处理流程

### 2.4 系统结构设计

### 2.5 功能需求与程序的关系

### 2.6 人工处理过程

### 2.7 尚未解决的问题

## 3 接口设计

### 3.1 用户接口

### 3.2 外部接口

### 3.3 内部接口

## 4 运行设计

### 4.1 运行模块组合

### 4.2 运行控制

### 4.3 运行时间

## 5 系统数据结构设计

5.1 逻辑结构设计要点

5.2 物理结构设计要点

5.3 数据结构与程序的关系

6 系统出错处理设计

6.1 出错信息

6.2 补救措施

6.3 系统维护设计

# 1 引言

本概要设计说明书用于描述分布式博客系统的总体设计方案、模块划分、系统运行流程以及数据库设计。本项目基于 `mogu_blog_v2` 项目进行二次开发，在原有博客系统基础上扩展 MongoDB 博客正文存储、MySQL 读写分离、Redis 高性能缓存、ElasticSearch 全文检索、RabbitMQ 异步消息处理等功能，目标是构建一个高并发、高可扩展、高可维护性的现代化博客平台。本文档主要服务于开发人员、测试人员、系统维护人员以及项目管理人员，为后续详细设计、编码实现、测试部署以及系统运维提供统一依据。

## 1.1 编写目的

随着互联网技术的发展，传统单体博客系统已经难以满足高并发、大数据量以及高扩展性的需求。为了提升系统整体性能与系统扩展能力，本项目采用 Spring Cloud Alibaba 微服务架构设计，通过服务拆分实现业务模块解耦，并通过 Redis、ElasticSearch、MongoDB 等技术提升系统性能与用户体验。本文档的编写目的在于明确系统总体设计方案、模块划分、数据流向以及关键技术实现方式，为项目开发团队提供统一的技术规范与设计依据。

## 1.2 范围

本系统名称为“分布式博客系统”。系统主要面向博客作者、普通用户以及后台管理员提供服务。系统支持博客文章发布、博客分类管理、标签管理、评论管理、用户登录注册、后台管理、全文检索、热点缓存、消息通知以及日志监控等功能。项目运行于 Linux 服务器环境，支持 Docker 容器化部署。开发团队负责系统开发、测试、部署与维护工作，最终用户包括博客作者、普通访客以及系统管理员。

## 1.3 定义

**MongoDB:** 一种面向文档存储的 NoSQL 数据库，用于存储博客正文等非结构化数据。

**Redis:** 高性能 Key-Value 内存数据库，用于缓存热点数据。

**ElasticSearch:** 基于 Lucene 的分布式全文检索引擎。

**RabbitMQ:** 消息队列中间件，用于异步任务处理。

**JWT:** JSON Web Token，用于用户身份认证。

**Nacos:** 服务注册与配置中心。

OpenFeign：声明式服务调用框架。

Docker：容器化部署平台。

## 1.4 参考资料

本项目参考资料包括：mogu\_blog\_v2 项目源码、Spring Cloud Alibaba 官方文档、MongoDB 官方文档、ElasticSearch 官方文档、Redis 官方文档、RabbitMQ 官方文档以及阿里巴巴 Java 开发手册等。系统设计遵循 RESTful API 风格规范，并结合微服务架构设计思想进行实现。

## 2 总体设计

系统采用 Spring Cloud Alibaba 微服务架构进行设计，前后端采用分离模式部署。前端基于 Vue 与 ElementUI 开发，后端采用 Spring Boot 构建微服务集群。系统整体划分为网关服务、认证服务、博客服务、搜索服务、评论服务、文件服务、消息服务以及后台管理服务等多个模块，各模块之间通过 OpenFeign 与 HTTP 协议进行通信。

### 2.1 需求规定

系统需要满足以下需求：

1. 支持用户注册、登录、JWT 鉴权与权限控制；
2. 支持博客发布、编辑、删除、分类管理与标签管理；
3. 支持 ElasticSearch 全文检索；
4. 支持 Redis 热点缓存；
5. 支持 MongoDB 博客正文存储；
6. 支持 MySQL 主从复制与读写分离；
7. 支持 RabbitMQ 异步消息通知；
8. 支持后台管理与日志监控；
9. 支持 Docker 容器化部署；
10. 支持高并发访问与服务扩展。

系统要求具有良好的稳定性、安全性以及可维护性，能够满足大量用户并发访问需求。

### 2.2 运行环境

系统运行环境主要包括以下内容：

1. 操作系统：Linux Ubuntu 22.04；

2. JDK 版本: JDK17;
3. 数据库: MySQL8、MongoDB6;
4. 缓存数据库: Redis7;
5. 搜索引擎: ElasticSearch8;
6. 消息队列: RabbitMQ;
7. 注册中心: Nacos;
8. Web 服务器: Nginx;
9. 容器平台: Docker 与 Docker Compose;
10. 开发工具: IDEA、Maven、Git。

服务器最低配置要求为 4 核 CPU、8GB 内存、100GB 磁盘空间, 生产环境推荐使用独立服务器部署 ElasticSearch 与 MongoDB。

## 2.3 基本设计概念和处理流程

系统采用典型的微服务架构设计。用户请求首先进入 Nginx 反向代理服务器, 随后由 Spring Cloud Gateway 网关进行统一路由转发。认证服务负责 JWT 鉴权与用户权限校验, 博客服务负责文章管理与业务逻辑处理。

当用户发布博客时:

1. 用户通过前端页面提交博客内容;
2. 网关服务转发请求至博客服务;
3. 博客元数据写入 MySQL;
4. 博客正文内容写入 MongoDB;
5. 系统通过 RabbitMQ 异步更新 ElasticSearch 索引;
6. Redis 更新热点缓存数据。

当用户搜索博客时:

1. 用户输入关键词;
2. 请求进入搜索服务;
3. ElasticSearch 返回匹配结果;
4. Redis 缓存热门搜索结果;
5. 最终返回博客列表。

系统通过 Redis 缓存机制降低数据库访问压力, 通过 MongoDB 提升非结构化文本存储能力, 通过 ElasticSearch 提升全文检索性能。

## 2.4 系统结构设计

系统整体结构主要包括以下模块：

1. 网关模块：

负责统一路由转发、权限校验、限流以及日志记录。

2. 认证模块：

负责用户登录、JWT 令牌生成、权限校验与用户认证。

3. 博客模块：

负责博客文章发布、编辑、删除、分类管理与标签管理。

4. 评论模块：

负责评论发布、评论审核、点赞与消息通知。

5. 搜索模块：

基于 Elasticsearch 实现全文检索与搜索推荐。

6. 文件模块：

负责图片上传、文件存储与访问控制。

7. Redis 缓存模块：

缓存热门博客、首页数据、标签数据以及热门搜索结果。

8. MongoDB 模块：

负责存储博客正文、评论内容等非结构化数据。

9. MySQL 数据库模块：

负责存储用户信息、博客元数据、标签、分类以及权限信息。

10. RabbitMQ 消息模块：

用于异步任务处理，包括索引更新、消息通知以及日志异步写入。

11. 后台管理模块：

管理员可通过后台系统完成博客审核、用户管理、权限管理以及系统监控。

## 2.5 功能需求与程序的关系

系统各项功能需求与程序模块关系如下：

1. 用户登录与权限控制由认证服务实现；
2. 博客管理功能由博客服务实现；
3. 搜索功能由 Elasticsearch 搜索服务实现；
4. Redis 缓存由缓存模块实现；
5. MongoDB 正文存储由博客内容模块实现；
6. RabbitMQ 异步消息由消息服务实现；
7. 后台管理功能由 Admin 服务实现。

系统通过微服务拆分实现业务模块解耦，提高系统扩展性与维护性。

## 2.6 人工处理过程

系统运行过程中需要管理员进行部分人工维护操作：

1. 定期执行 MongoDB 数据备份；
2. 定期重建 Elasticsearch 索引；
3. 定期清理 Redis 缓存；
4. 检查 RabbitMQ 消息积压情况；
5. 审核用户评论与博客内容；
6. 更新系统配置与安全策略。

系统管理员需要具备 Linux 运维能力以及数据库维护能力。

## 2.7 尚未解决的问题

当前系统仍存在以下尚未完全解决的问题：

1. Elasticsearch 在超高并发场景下的性能优化；
2. MongoDB 分片集群部署方案；
3. 微服务分布式事务一致性问题；
4. Redis 集群高可用部署；
5. 系统日志集中化管理与链路追踪；
6. AI 智能推荐功能与向量检索功能扩展。

## 3 接口设计



系统接口设计主要包括用户接口、外部接口与内部接口。系统通过 RESTful API 提供服务接口，并通过 JSON 格式进行数据传输。

### 3.1 用户接口

系统前端采用 Vue + ElementUI 开发，主要页面包括：首页、博客详情页、分类页面、标签页面、登录注册页面、用户中心页面以及后台管理页面。用户通过浏览器访问系统，系统采用响应式布局设计，支持 PC 与移动端访问。后台管理页面支持博客审核、评论管理、用户管理、权限配置以及系统监控等功能。

### 3.2 外部接口

系统与外部中间件接口如下：

1. MySQL 数据接口；
2. MongoDB 数据接口；
3. Redis 缓存接口；
4. Elasticsearch 搜索接口；
5. RabbitMQ 消息接口；
6. Nacos 服务注册接口；
7. 文件存储接口。

系统采用 Spring Data MongoDB 访问 MongoDB，采用 MyBatis Plus 操作 MySQL，采用 Spring Data Redis 操作 Redis。

### 3.3 内部接口

系统内部微服务之间通过 OpenFeign 与 HTTP 接口通信。服务调用包括用户服务调用、博客服务调用、搜索服务调用以及评论服务调用。系统通过统一返回结果格式实现接口规范化，返回数据统一采用 JSON 格式。

## 4 运行设计

系统采用 Docker 容器化运行模式，各微服务独立部署并通过 Nacos 完成服务注册。

### 4.1 运行模块组合

系统运行时主要模块包括：

1. Nginx 网关模块；
2. Gateway 网关服务；

3. Auth 认证服务；
4. Blog 博客服务；
5. Search 搜索服务；
6. Comment 评论服务；
7. Admin 后台管理服务；
8. Redis 缓存服务；
9. MongoDB 数据服务；
10. MySQL 主从数据库；
11. ElasticSearch 搜索引擎；
12. RabbitMQ 消息队列。

各模块协同工作完成博客系统整体业务流程。

## 4.2 运行控制

系统采用 Docker Compose 与 Kubernetes 进行部署管理。系统启动流程如下：

1. 启动 MySQL 与 MongoDB；
2. 启动 Redis 与 RabbitMQ；
3. 启动 ElasticSearch；
4. 启动 Nacos 注册中心；
5. 启动各业务微服务；
6. 启动 Nginx 网关。

系统通过 Nacos 动态管理配置文件与服务注册信息。

## 4.3 运行时间

系统性能要求如下：

1. 普通页面响应时间低于 1 秒；
2. ElasticSearch 搜索响应时间低于 500ms；
3. Redis 缓存读取时间低于 50ms；
4. 博客发布操作响应时间低于 2 秒；
5. 系统支持至少 1000 用户并发访问。

## 5 系统数据结构设计

系统采用 MySQL + MongoDB 混合存储架构，实现结构化数据与非结构化数据分离存储。

## 5.1 逻辑结构设计要点

MySQL 数据表包括：

1. 用户表；
2. 角色权限表；
3. 博客元数据表；
4. 标签表；
5. 分类表；
6. 评论索引表；
7. 日志表。

MongoDB 集合包括：

1. 博客正文集合；
2. 评论内容集合；
3. 操作日志集合。

ElasticSearch 索引包括博客标题、摘要、标签以及正文关键字。

## 5.2 物理结构设计要点

MySQL 采用主从复制实现读写分离，主库负责写操作，从库负责读操作。

MongoDB 使用 BSON 文档结构存储博客正文。Redis 使用 String、Hash 与 ZSet 等结构缓存热点数据。ElasticSearch 使用倒排索引提升全文搜索效率。

## 5.3 数据结构与程序的关系

博客服务访问 MySQL 与 MongoDB；搜索服务访问 ElasticSearch；缓存服务访问 Redis；认证服务访问 MySQL 用户表。各模块通过统一数据接口进行数据访问。

## 6 系统出错处理设计

系统需要具备良好的异常处理与故障恢复能力。

### 6.1 出错信息

系统在出现数据库连接失败、Redis 缓存失效、ElasticSearch 节点异常、MongoDB 写入失败等问题时，统一输出错误日志与异常提示信息。系统采用全局异常处理机制记录错误信息，并通过日志系统输出详细错误堆栈。

## 6.2 补救措施

系统采用以下故障恢复方案：

1. Redis 缓存降级；
2. ElasticSearch 索引重建；
3. MongoDB 数据备份恢复；
4. MySQL 主从切换；
5. RabbitMQ 消息重试机制；
6. Docker 容器自动重启。

系统通过 Sentinel 实现服务熔断与限流，保证系统在高并发情况下仍具备稳定性。

## 6.3 系统维护设计

系统采用统一日志管理、Docker 容器化部署以及自动化监控方案提高系统维护效率。系统通过 Spring Boot Admin、Prometheus 与 Grafana 实现系统监控与性能分析。管理员可实时查看系统运行状态、CPU 使用率、内存占用情况以及接口响应时间。